

Activity 14 - Design and document a reorder feature with AI

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K5/A5 K4/A4

Specification

`reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU.

Plan before code

Act as a senior Rust reviewer. Specification: `reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 order_qty = point + 7 days usage - on_hand, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Design and document a reorder feature with AI" in starter/src so that the supplied tests pass. Give the AI R1-R4 and the public types; require a trace table in its answer and verify each row against the tests. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.